CSE276A HW1 Report: Waypoint Navigation

Fong-Yu Lin · Oct 12, 2026

YouTube Video

https://youtu.be/189XVaLgCpw?si=ek8AVvpiOvj4hXzN

Summary

My rover drove all 7 waypoints on its own, with every final heading within 4.1° of the target.

The key idea: the robot can **measure which way it's pointing** (with the gyro), but it **can't measure where it is**. So I close the loop on heading and use calibrated timing for distance:

- **Heading: closed-loop.** The gyro measures turn rate, I add it up to get the heading, and the robot keeps correcting until the heading error is small.
- **Distance: open-loop.** The robot drives for `distance / speed` seconds and assumes it arrived. Nothing checks how far it really went.

Getting there took some debugging. The IMU's built-in yaw was unreliable while turning, and the IMU data was arriving in bursts, which broke my heading math until I fixed it.

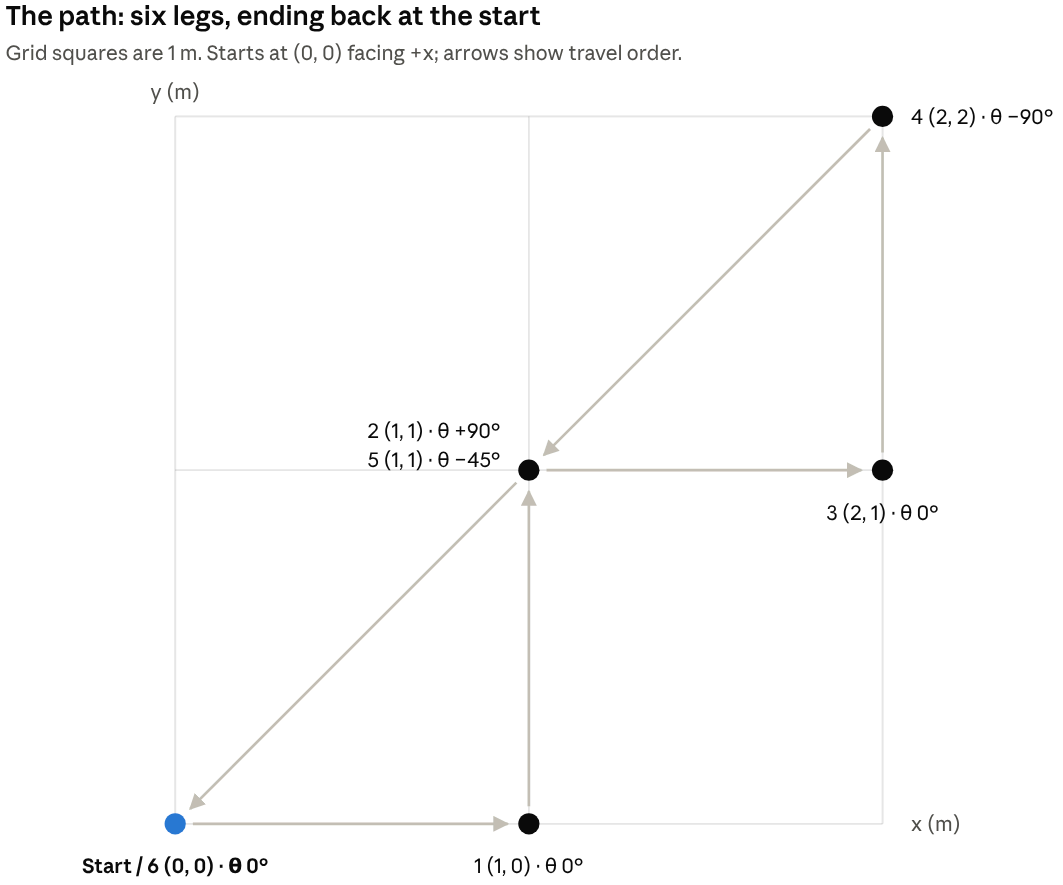
The task

The robot starts at (0, 0) facing +x and has to visit 6 waypoints in order, ending back at the start. Each waypoint is `(x, y, θ)`: a position in meters and the heading the robot should face when it gets there, in radians.

Convention: +x is the robot's starting forward direction, +y is to its left, and angles are counter-clockwise positive (so +90° = facing left of the start, −90° = facing right).

#	x (m)	y (m)	θ (rad)	θ (deg)
start	0	0	0	0
1	1	0	0	0
2	1	1	1.57	+90

#	x (m)	y (m)	θ (rad)	θ (deg)
3	2	1	0	0
4	2	2	−1.57	−90
5	1	1	−0.78	−45
6	0	0	0	0



waypoint path · 6 legs, 1 m grid

The last two legs run back down the same diagonal, (2, 2) → (1, 1) → (0, 0), and (1, 1) is visited twice with different headings.

The robot. A RubikPi rover running Ubuntu 24.04 and ROS 2 Jazzy. It's differential drive: one command for the left wheels (`L`) and one for the right (`R`), each from −0.5 to 0.5. Same values = straight, opposite values = spin in place. It can't move sideways, so every leg has to be a turn plus a straight line. The commands go over serial to a motor board, which also sends back IMU data (including a gyro).

How it works

Each leg is the same three steps: turn to face the next waypoint, drive straight to it, then turn to the required heading.

1. **Turn** in place until facing the next waypoint.
2. **Drive** straight for the distance between the two waypoints.
3. **Turn** in place to the waypoint's θ, then pause 1 s.

The direction and distance to the next waypoint come from basic geometry:

$$\text{direction} = \operatorname{atan2}(\Delta y,\ \Delta x), \qquad \text{distance} = \sqrt{\Delta x^2 + \Delta y^2}$$

where Δx and Δy are how far the next waypoint is from the current one (m). `atan2(y, x)` is the arctangent that looks at the signs of x and y separately, so it gives the right direction in all four quadrants. Plain `atan(y/x)` can't: (1, 1) and (−1, −1) both give y/x = 1.

When the robot turns, it should take the short way around. I wrap every angle difference into (−180°, 180°]:

$$\Delta\theta = \operatorname{atan2}\big(\sin(\theta_{target} - \theta),\ \cos(\theta_{target} - \theta)\big)$$

where θ is the current heading and θ_target is the one I want. For example, going from 170° to −170°, plain subtraction says turn −340°, but wrapping gives +20°.

Applied to this path:

Leg	Turn 1	Drive	Turn 2
start → 1	0°	1 m	0°
1 → 2	left 90°	1 m	0°
2 → 3	right 90°	1 m	0°
3 → 4	left 90°	1 m	180°
4 → 5	right 45°	1.414 m	left 90°
5 → 6	right 90°	1.414 m	left 135°

What's measured vs. assumed. This is the most important thing to understand about my approach:

Quantity	How the robot knows it	Feedback?
Heading	Gyro turn rate, added up over time	Yes: corrects until the error is small
Distance driven	Time × calibrated speed	No: just drives for a set time
Position	Assumed to be the waypoint it was aiming for	No

After each leg, the robot treats the target waypoint as its new position and plans the next leg from there. If a leg comes up 5 cm short, the robot never finds out, and the error carries into every leg after.

Why turn-drive-turn? Calibration showed this robot can't turn slowly (it either doesn't move or spins fast). A controller that blends turning and driving into smooth curves would be hard to tune with that. Keeping turning and driving separate made each one easy to get right.

Calibration

Calibration gave me four facts the controller depends on:

What	Result	Used for
Driving speed	command 0.3 → 0.377 m/s	how long to drive each leg
Deadband	below ~0.07 the wheels don't move	keeping commands high enough
Turning	only reliable at command 0.5 (~190°/s)	turn command
Heading sensor	built-in IMU yaw is wrong during turns; raw gyro is accurate	heading estimate

To measure these I wrote a small helper (`calibrate.py`): it sends a fixed `(L, R)` for a fixed time, stops, and prints how much the robot turned. I measured distance with a tape measure and angles by eye against tape on the floor.

1. Driving speed

Command (L = R)	Time	Distance, 3 runs	Speed
0.2	2 s	42, 42.5, 43 cm	0.213 m/s
0.3	2 s	76, 75, 75 cm	0.377 m/s
0.3	2.65 s	100 cm	0.377 m/s (check)

The runs were very repeatable (within 1 cm). Speed isn't simply proportional to the command, though. Fitting a line through the two speeds:

```
v \approx 1.64 \cdot \text{cmd} - 0.116
```

where v is the forward speed (m/s) and cmd is the command on both wheels. The negative intercept is friction: setting v = 0 gives cmd ≈ 0.07, and that's exactly where the robot stopped moving in my tests (0.07 didn't move, 0.08 barely did). This dead zone is the **deadband**.

I chose 0.3 for driving: well above the deadband, not too fast. Driving 2.65 s at 0.3 landed exactly on 1 m.

One more thing: open-loop at `L = R = 0.3`, the robot slowly drifts left, so the right wheel is slightly faster. I handle that with heading feedback instead of trimming the wheels (see Control).

2. Turning

Command	Type	What happened
−0.3, 0.3	spin in place	barely moves
−0.35, 0.35	spin in place	sometimes moves, sometimes not
−0.4 / −0.45	spin in place	turns, but often won't start from standstill
−0.5, 0.5	spin in place	~190° in 1 s, every time
0, 0.4	one wheel only	doesn't move
0, 0.5	one wheel only	~180° in 2 s, every time

Turning is all-or-nothing. To spin in place the wheels have to skid sideways, which takes a lot of force to start. Once it starts it spins fast. So there's no gentle turn available, and I use 0.5 for all turns.

3. Which sensor to trust for heading

The motor board reports a yaw angle, but it was way off during turns:

Test	Board yaw	My gyro integration	Actual
spin 0.5, 1 s	502°	—	~190°
one wheel 0.5, 2 s	348°, then 19° (same test!)	—	~180°
one wheel 0.5, 2 s	355°	194°	~180°

I logged the raw IMU data to see why. Standing still, yaw is stable. During a turn, the gyro reads a steady ~100°/s, but yaw speeds up, jumps around, and after stopping it drifts back ~30° by itself. My guess is the board mixes in a magnetometer, which the motors disturb. So I ignore yaw and compute heading from the raw gyro myself.

4. Is the gyro accurate?

I spun about one full turn (one wheel at 0.5 for 4 s) three times and compared to the floor tape:

Run	Gyro said	Actual
1	369°	matched
2	377°	matched
3	379°	matched

The gyro is accurate, so no scale correction is needed. It does have a small offset when still (about −0.03 to −0.6°/s depending on the run), which I measure at startup and subtract.

Control

The controller has three parts: a heading estimate from the gyro, a turning controller, and a driving controller. Both controllers use the same heading estimate as feedback.

1. Heading estimate

Idea: the gyro tells me how fast the robot is turning, so heading = add up (turn rate × time), like distance = speed × time.

At startup the robot stays still for 2 s and I average the gyro reading to get its offset (bias). Then every IMU sample (~48 per second) updates the heading:

```
\theta \leftarrow \operatorname{wrap}\big(\theta + (g_z - b)\,\Delta t\big)
```

where:

- θ = heading (rad), ← means "replace with"
- g_z = gyro reading, the turn rate (rad/s), counter-clockwise positive
- b = bias, the gyro reading while still (should be 0, so whatever it reads is error)
- Δt = time per sample, about 0.02 s
- wrap = keeps the angle in (−180°, 180°], from How it works

The Δt bug. At first I used the time between messages as Δt. It turned out messages arrive in bursts: 4–5 at almost the same instant, then nothing for ~0.09 s. So the first message in each burst got all the time and the rest got almost none. On long turns this averaged out, but on short correction pulses the heading came out way off (my first turn test ended ~50° wrong).

The cause was in the template `motor_control.py`: it read serial with `read(1024)` and a 0.1 s timeout, so it only handed off data every 0.1 s. I fixed it two ways:

- `motor_control.py` now reads whatever is available right away (`read(in_waiting or 1)`). Data arrives on time, which also removes ~0.1 s of delay from the control loop.
- Δt is a running average of the time between samples, so even if bursts happen it stays correct:

```
\Delta t \leftarrow 0.98\,\Delta t + 0.02\,\text{gap}
```

where gap is the time since the previous message. Each new gap only counts 2%, so Δt is roughly the average of the last 50 samples.

2. Turning

Idea: since the robot can only turn at full power, turn at full power and stop a bit early, then nudge.

1. Spin at 0.5 toward the target until the remaining error is under 15°. Stopping early makes up for delay and coasting.
2. If the error is still over 5°, nudge with a 0.12 s pulse, wait 0.3 s, check again.
3. Stop after 8 tries so it can't get stuck.

The pulse size and the tolerance have to match. If one pulse turns more than the tolerance, the robot overshoots back and forth forever. With 4° tolerance and 0.15 s pulses it did exactly that; 5° and 0.12 s fixed it.

3. Driving straight

Idea: while driving, if the robot drifts off its heading, speed up one wheel and slow down the other, more for a bigger drift.

This is a P (proportional) controller:

```
e = \operatorname{wrap}(\theta_{target} - \theta), \qquad c =
\operatorname{clip}(K_p\, e,\ \pm 0.1), \qquad L = 0.3 - c, \quad R = 0.3 + c
```

where:

- e = heading error (rad), positive = robot is pointing too far right
- θ_target = the direction to the next waypoint
- K_p = proportional gain, how hard to correct per radian of error (0.3)
- c = correction; clip limits it to ±0.1 so both wheels stay well above the deadband
- L, R = left and right wheel commands, around the base speed 0.3

Example: drifting 10° (0.17 rad) to the left gives e = −0.17, c = −0.05, so L = 0.35 and R = 0.25, which turns it back right. This is what fixed the left drift from calibration, without me needing to know how much faster the right wheel is. The robot drives for distance / 0.377 seconds.

Parameters

Parameter	Value	Why
Drive command	0.3	well above the 0.07 deadband
Drive speed	0.377 m/s	measured (1 m in 2.65 s)
K_p	0.3 per rad	holds heading without wobbling
Max correction	0.1	keeps both wheels moving
Turn command	0.5	0.45 often won't start
Turn stop-early angle	15°	fast turn + delay
Turn tolerance	5°	must be bigger than one nudge (4° oscillated)
Nudge length	0.12 s	0.1 s didn't move it, 0.15 s overshot
Bias measurement	2 s	

Code

The code is a ROS 2 package (`robot_control`) with two nodes running at the same time. My navigator decides the wheel commands; the motor node talks to the hardware.

```
waypoint_navigator  --/motor_commands [L, R]-->  motor_control  --serial-->
motor board
        ^                                                |
        +------------------- /imu_data -----------------+
```

File	What it does
`waypoint_navigator.py`	**My main node.** Measures gyro bias, tracks heading, and runs turn → drive → turn for each waypoint (`turn_to()`, `drive_straight()`).
`motor_control.py`	From the course template. Forwards `[L, R]` to the motor board at 20 Hz, publishes IMU data, and stops the motors if no command arrives for 0.15 s. I fixed its serial read.

File	What it does
`calibrate.py`	My calibration helper: drive `(L, R)` for N seconds, print the turn angle, save raw IMU data.
`ssh_keyboard_control.py`	Keyboard driving over SSH (the template version needs a screen, so it doesn't work over SSH).
`waypoints/*.txt`	Waypoint lists, one `x,y,theta` per line: `hw1.txt`, plus `test_turns.txt` and `test_first_leg.txt` for testing.

To run on the rover:

```
ros2 run robot_control motor_control                # terminal 1
ros2 run robot_control waypoint_navigator hw1.txt   # terminal 2
```

Results

The full run reached every waypoint with heading error under 4.1°, and position error was acceptable on the floor markers.

I didn't run the whole path first. Errors pile up from leg to leg, so I tested one piece at a time: one straight leg, then four turns, then everything.

Step-by-step tests

#	Test	Settings	Result	Next change
1	1 m straight		1.00 m, very straight, heading off by 2.8°	none needed
2	4 × left 90°	turn 0.45, nudge 0.1 s, old Δt	turns stopped short (+24°, +12° error); ended ~50° off for real even though the gyro said +4°	turn 0.5, nudge 0.15 s, fixed Δt bug
3	4 × left 90°	nudge 0.15 s, tolerance 4°	each turn within 1.5°, but nudges overshot back and forth	nudge 0.12 s, tolerance 5°

#	Test	Settings	Result	Next change
4	4 × left 90°	nudge 0.12 s, tolerance 5°	each turn within 2.5°, no back-and-forth, ended back at the start heading	final settings

Test 2 is where the Δt bug showed up: I had already confirmed the gyro itself was accurate, so a 50° mismatch had to come from how I was adding it up.

Full run

Heading at each waypoint (from the robot's estimate):

Waypoint	Target	Reached	Error
1 (1, 0)	0°	+2.9°	−2.9°
2 (1, 1)	+90°	+93.1°	−3.1°
3 (2, 1)	0°	+2.9°	−2.9°
4 (2, 2)	−90°	−94.0°	+4.1°
5 (1, 1)	−45°	−45.4°	+0.7°
6 (0, 0)	0°	−4.0°	+4.0°

Total driving time was 18.1 s (four 1 m legs at 2.65 s, two 1.414 m legs at 3.75 s), plus turns and pauses.

One pattern stands out: after each of the first three straight legs, the robot ended up about 3° to the left. That's under the 5° tolerance, so it never got corrected. I explain why in the Discussion.

Discussion

The main limitation is that position is never measured, so distance errors add up with nothing to correct them.

Limitations

- **Open-loop distance.** With no wheel encoders or external sensor, anything that changes the real speed (battery level, floor surface, wheel slip) shows up directly as position error, and it carries into every later leg. The last two legs are the longest (1.414 m) and come after four legs of built-up error, so the return to (0, 0) is the least accurate.
- **Coarse turning.** The robot only turns reliably at full power (~190°/s), so it can't stop exactly on target. That's why I need a 5° tolerance.
- **Battery.** I calibrated speed at one charge level. On a lower battery, 2.65 s might not be 1 m anymore.

Why there's a steady 3° drift: P vs. PI control

My driving controller is a P controller: correction = K_p × error. It only pushes back when there **is** an error. The right wheel is always a bit faster, which acts like a constant push to the left. The robot settles at the point where the P correction exactly cancels that push, and to make any correction it needs some error. Using the ~3° (0.052 rad) I saw:

```
c = K_p\, e = 0.3 \times 0.052 \approx 0.016, \qquad R - L = 2c \approx 0.03
```

where c is the correction, and R − L is the resulting difference between the wheel commands (L gets −c, R gets +c). So the right wheel is about 0.03 faster than the left at a command of 0.3, roughly 10%, and the controller has to sit 3° off to cancel it. This leftover error is called **steady-state error**. A bigger K_p shrinks it but never to zero, and too big makes the robot wobble.

Adding an integral term (**PI control**) would remove it:

```
c = K_p\, e + K_i \int e\, dt
```

where:

- K_p e = the P term, reacts to the error right now
- ∫ e dt = the error added up over time since the leg started (rad·s)
- K_i = integral gain, how fast that built-up error turns into correction

As long as the error stays on one side, the integral keeps growing and pushes harder, until the error is gone. Then the P term is zero, but the integral holds the value that cancels the faster wheel. In other words, it learns the wheel mismatch on its own. I'd reset the integral at the start of each leg and cap it so it can't build up too much (integral windup). A first

guess is K_i ≈ 0.3, which builds the needed 0.016 in about 1 s. A D term (full PID) reacts to how fast the error changes and reduces overshoot, but the driving doesn't oscillate, so I don't think it's needed here.

What I'd improve

- Add the integral term to the driving controller and compare against P only on the 1 m test.
- Use wheel encoders, if the motor board reports them, to measure distance instead of timing it.
- Use AprilTags (the `apriltag_ros` package is already in the repo) to correct position against known landmarks.